



Alzheimer's sMRI / fMRI Data Preprocessing & Analysis (fMRIPrep + Python)

Overview

Students will build a reproducible neuroimaging pipeline to preprocess Alzheimer's sMRI / fMRI data (structural T1-weighted and — if present — functional BOLD) using **fMRIPrep**, then perform downstream MRI analyses using Python. The assignment emphasizes reproducibility, BIDS organization, containerized processing, quality control, feature extraction, statistical modeling, and machine learning classification.

Learning objectives

By the end of this assignment students will be able to:

1. Convert raw DICOMs to BIDS and validate dataset compliance.
 2. Run fMRIPrep (Docker/Singularity) with appropriate options and FreeSurfer integration.
 3. Inspect fMRIPrep outputs and perform QC.
 4. Extract structural biomarkers (volumes, cortical thickness) and parse FreeSurfer outputs.
 5. Perform voxelwise / regionwise analyses and run group-level statistics with Python (nilearn, statsmodels).
 6. Build a simple ML classifier (scikit-learn) to distinguish AD vs CN using extracted features.
 7. (Optional) Run resting-state functional connectivity analyses using outputs from fMRIPrep.
-

Dataset

Primary dataset: ADNI (Alzheimer's Disease Neuroimaging Initiative)

- Students must obtain ADNI access (registration required). Use a subset to keep compute reasonable — e.g., 50 subjects balanced across diagnostic groups (AD, MCI, CN).
- If ADNI access is an issue, provide a small open dataset alternative (e.g., OASIS-3 subset, or a synthetic/BIDS sample dataset).

Dataset selection & labels:

- For supervised tasks, collect demographics and diagnostic labels (AD, MCI, CN) in a `participants.tsv` BIDS file or separate CSV.
-

Prerequisites

- Linux-based workstation or cloud VM (8+ CPU cores, 32+ GB RAM recommended). GPU not required for fMRIPrep but helpful for downstream DL experiments.
- Basic familiarity with bash, Python, neuroimaging file formats (NIfTI), BIDS spec.
- FreeSurfer license file (for FreeSurfer recon-all if using `--fs-license-file`).

Software & packages

System / container

- Docker. fMRIPrep image used from `nipreps/fmriprep`.
- FreeSurfer (optional if using `--fs-license-file` and enabling surface reconstructions).

Python packages (conda/pip)

- `nibabel`, `nilearn`, `nilearn.datasets`, `numpy`, `pandas`, `matplotlib`, `seaborn` (optional), `scikit-learn`, `statsmodels`, `scipy`, `joblib`.
- Optional: `nipyype`, `nilearn.decomposition` (ICA), `brainiak` or `bctpy` for graph metrics.

Deliverables

1. BIDS-organized dataset (or link/description of subset used).
2. A reproducible preprocessing script (bash) that runs fMRIPrep with clear flags.
3. Python notebooks/scripts for: feature extraction, QC visualizations, statistical models, and ML classifier.
4. A short report (PDF/Markdown) describing methods, results (figures/tables), limitations.
5. A `README.md` with instructions to reproduce the work.

Project structure (recommended)

```
project/
├─ rawdata/           # original DICOMs or BIDS input
├─ bids/              # converted BIDS dataset (output of
dcm2bids/heudiconv)
├─ derivatives/
│   ├─ fmriprep/      # fMRIPrep outputs
│   ├─ freesurfer/    # if using recon-all
│   └─ custom/        # outputs of your analysis (masks, connectivity
matrices)
├─ scripts/
│   ├─ run_fmriprep.sh
│   └─ analysis/
│       ├─ extract_volumes.py
│       ├─ connectivity_analysis.py
│       └─ classification_pipeline.ipynb
├─ notebooks/
├─ results/
└─ README.md
```

Part A — BIDS conversion and validation (10 points)

Tasks:

1. Convert raw DICOMs to BIDS using `heudiconv` or `dcm2bids`. Provide and explain the heuristic used (if `heudiconv`). Example `dcm2niix` or `heudiconv` commands should be included.
2. Run `bids-validator` and fix any issues.

Example (heudiconv):

```
heudiconv -d '/path/to/dicoms/{subject}/**/*.dcm' -s sub-001 -f
convert_heuristic.py -o ./bids -b
```

Example (bids-validator):

```
npx bids-validator ./bids
```

Part B — Preprocessing with fMRIPrep (30 points)

Goal: Run fMRIPrep to preprocess T1w and BOLD images, produce normalized outputs, and generate confounds and QC HTML.

Key points to cover in report:

- Explain each major output folder (e.g., `anat/`, `func/`, `freesurfer/`, `figures/`).
- Explain confounds TSV (head motion parameters, `aCompCor`, framewise displacement, etc.).

Example fMRIPrep Docker command (adapt for your system):

```
docker run --rm -it \
-v /path/to/bids:/data:ro \
-v /path/to/derivatives:/out \
-v /path/to/freesurfer/license.txt:/opt/freesurfer/license.txt:ro \
nipreps/fmriprep:latest /data /out participant \
--participant-label 001 002 003 \
--fs-license-file /opt/freesurfer/license.txt \
--output-spaces MNI152NLin2009cAsym:res-2 \
--nthreads 8 --mem_mb 30000 \
--ignore fieldmaps
```

Notes:

- If dataset lacks fieldmaps, set `--ignore fieldmaps` or enable sinc-based unwarping with `--use-syn-sdc` (if appropriate).
- Choose output space(s) carefully — `MNI152NLin2009cAsym` is a common choice.

QC:

- Inspect `derivatives/fmriprep/sub-*/figures/sub-*_task-*_bold_confounds.html` and `fmriprep` generated reports. Document issues (e.g., bad motion).
 - Compute summary motion stats across subjects using Python.
-

Part C — Structural analysis & feature extraction (25 points)

1. FreeSurfer-derived features

- From `derivatives/freesurfer` or `fMRIPrep`'s `recon-all` outputs, extract volumes (e.g., hippocampus, ventricles) from `aseg.stats` or `aparc.stats`.

Example Python to parse `aseg.stats` (skeleton)

```
import pandas as pd
import glob

files = glob.glob('derivatives/freesurfer/*/stats/aseg.stats')
rows = []
for f in files:
    subj = f.split('/')[-3]
    with open(f) as fh:
        for line in fh:
            if line.startswith('#'):
                continue
            parts = line.strip().split()
            if len(parts) >= 5:
                # freesurfer aseg format: struct_id, seg_id, NVoxels,
                # Volume_mm3, StructName
                struct_name = parts[-1]
                vol_mm3 = float(parts[3])
                rows.append({'subject': subj, 'structure': struct_name,
                           'volume_mm3': vol_mm3})

df = pd.DataFrame(rows)
df_pivot = df.pivot(index='subject', columns='structure',
                    values='volume_mm3')
df_pivot.to_csv('results/aseg_volumes.csv')
```

2. Voxel-based morphometry (optional)

- Use smoothed modulated gray-matter maps (you may use SPM/FSL outside scope) or conduct regionwise analyses using atlas (e.g., AAL, Harvard-Oxford) with `nilearn` to compute mean intensities.

3. Statistical comparisons

- Run group-level comparisons (e.g., hippocampal volume AD vs CN) using `statsmodels` or `scipy.stats` (t-test, ANCOVA controlling for age, ICV).

Example:

```
import statsmodels.api as sm
from statsmodels.formula.api import ols

# df_combined has columns: volume, diagnosis (AD/CN), age, sex, ICV
model = ols('volume ~ C(diagnosis) + age + sex + ICV',
            data=df_combined).fit()
print(model.summary())
```

Part D — Machine Learning classification (20 points)

Tasks:

1. Build a classification pipeline to predict diagnostic label (AD vs CN) using structural features (volumes, cortical thickness, demographics).
2. Steps to include: feature selection/engineering, scaling, nested cross-validation (outer CV for performance, inner CV for hyperparameter tuning), report metrics (accuracy, precision, recall, ROC AUC), and feature importance.

Skeleton code (scikit-learn)

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold, cross_validate

X = features.values
y = labels.values
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', RandomForestClassifier(n_estimators=200, random_state=42))
])
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scoring = ['accuracy', 'roc_auc']
res = cross_validate(pipe, X, y, cv=cv, scoring=scoring,
return_estimator=True)
print({k: v.mean() for k, v in res.items() if 'test_' in k})
```

Deliverables: CSV of features, notebook showing CV results, confusion matrix, ROC curve.

Part E — Functional analyses (optional/extra credit, 20 points)

If the dataset includes fMRI:

1. Denoising & confounds

- Use fMRIPrep confounds (*desc-confounds_regressors.tsv) — choose confounds (e.g., aCompCor components, motion parameters, global signal if desired), create a denoised timeseries using Nilearn's `signal.clean`.

2. Seed-based connectivity

- Extract seed timeseries (e.g., PCC for default mode network) and compute correlation maps; transform to Fisher z and run group-level contrasts with Nilearn's `SecondLevelModel` or simple mass-univariate tests.

3. ICA / Network analysis

- Run group ICA (Nilearn.decomposition.CanICA) to extract networks and compare component maps between groups.

4. Functional connectivity matrices & graph metrics

- Extract parcelwise timeseries (using an atlas), compute correlation matrices, compute graph metrics (degree, modularity).

Skeleton for seed correlation:

```
from nilearn.input_data import NiftiMasker
```

```
from nilearn import image, plotting
import numpy as np

seed_mask_img = image.load_img('masks/pcc_seed.nii.gz')
masker = NiftiMasker(mask_img=seed_mask_img, standardize=True)
timeseries = masker.fit_transform('sub-01_task-rest_space-
MNI152NLin2009cAsym_desc-preproc_bold.nii.gz')
seed_ts = timeseries.mean(axis=1)
# Now compute voxelwise correlation with whole-brain
wb_masker = NiftiMasker(mask_img='masks/brain_mask.nii.gz', standardize=True)
wb_ts = wb_masker.fit_transform('...')
corr = np.dot(wb_ts.T, seed_ts) / (wb_ts.shape[0]-1)

# backproject to image
corr_img = wb_masker.inverse_transform(corr)
plotting.plot_stat_map(corr_img, title='Seed correlation (PCC)')
```

Part F — Quality control & reporting (10 points)

- Provide QC figures: motion summary plots, sample pre/post denoised timeseries, example confound plots from fMRIPrep reports.
 - Describe any excluded subjects and reasons.
 - Document any deviations from the planned pipeline.
-

Evaluation / Grading rubric

- BIDS conversion & validation: 10 points
- fMRIPrep run / reproducibility: 30 points
- Structural feature extraction & stats: 25 points
- ML classification pipeline & reporting: 20 points
- QC, documentation, reproducibility, code quality: 15 points

Total: 100 points. (Some split into sub-items above; rubric may be adapted.)

Appendix: quick checklist for students

1. Register and download a small ADNI subset (or use provided open dataset).
 2. Convert to BIDS and validate.
 3. Run fMRIPrep and inspect HTML reports.
 4. Extract FreeSurfer volumes and compute group differences.
 5. Build a classification pipeline and report metrics.
 6. (Optional) Run functional connectivity analyses.
 7. Submit notebooks, scripts, README.md, and a short report.
-

Starter scripts and templates

Below are starter scripts you can copy into `scripts/` to get the pipeline running quickly. They are intentionally minimal and commented for students.

1) `scripts/run_fmriprep.sh`

```
#!/usr/bin/env bash
# Run fMRIPrep via Docker for a small participant subset
# Usage: bash run_fmriprep.sh sub-001 sub-002

set -euo pipefail
```

```

BIDS_DIR="/path/to/bids"           # change to your bids directory
OUT_DIR="/path/to/derivatives"     # change to your derivatives output dir
FS_LICENSE="/path/to/license.txt"  # change to your FreeSurfer license (if
available)

# Participants provided as script args, or run all if none provided
if [ "$#" -ge 1 ]; then
    PARTICIPANTS=("$@")
else
    echo "No participants specified. Running all participants (may be slow)."
```

```

    PARTICIPANTS=( )
fi

# Build participant label flags
PART_FLAGS=""
for p in "${PARTICIPANTS[@]}"; do
    PART_FLAGS+=" --participant-label ${p}"
done

# Docker command (adapt volumes & resources for your machine)
docker run --rm -it \
    -v ${BIDS_DIR}:/data:ro \
    -v ${OUT_DIR}:/out \
    -v ${FS_LICENSE}:/opt/freesurfer/license.txt:ro \
    nipreps/fmriprep:latest /data /out participant \
    ${PART_FLAGS} \
    --fs-license-file /opt/freesurfer/license.txt \
    --output-spaces MNI152NLin2009cAsym:res-2 \
    --nthreads 8 --mem_mb 30000 \
    --ignore fieldmaps

# End

```

2) scripts/extract_volumes.py

```

#!/usr/bin/env python3
"""
Extract FreeSurfer aseg volumes from derivatives/freesurfer and save a CSV.
Usage: python scripts/extract_volumes.py --freesurfer_dir
derivatives/freesurfer --out results/aseg_volumes.csv
"""
import argparse
import glob
import os
import pandas as pd

parser = argparse.ArgumentParser()
parser.add_argument('--freesurfer_dir', required=True)
parser.add_argument('--out', required=True)
args = parser.parse_args()

rows = []
pattern = os.path.join(args.freesurfer_dir, 'sub-*', 'stats', 'aseg.stats')
for fpath in glob.glob(pattern):
    subj = os.path.basename(os.path.dirname(os.path.dirname(fpath)))
    with open(fpath, 'r') as fh:
        for line in fh:
            if line.strip() == '' or line.startswith('#'):
                continue
            parts = line.strip().split()
            # freesurfer aseg format (typical): Index Label NVoxels
            Volume_mm3 StructName
            if len(parts) >= 5:
                try:
                    vol_mm3 = float(parts[3])
                except ValueError:
                    continue
                struct_name = parts[-1]
                rows.append({'subject': subj, 'structure': struct_name,
                    'volume_mm3': vol_mm3})

if not rows:
    raise SystemExit('No aseg.stats files found. Check freesurfer_dir path
and that recon-all finished.')
```

```
df = pd.DataFrame(rows)
df_pivot = df.pivot(index='subject', columns='structure',
values='volume_mm3')
df_pivot.to_csv(args.out)
print(f"Wrote {args.out} with shape {df_pivot.shape}")
```

3) scripts/classification_pipeline.py

```
#!/usr/bin/env python3
"""
Simple ML pipeline using extracted features to classify AD vs CN.
Assumes a CSV with subjects as rows and features as columns, plus a
'diagnosis' column.
"""
import argparse
import pandas as pd
from sklearn.model_selection import StratifiedKFold, cross_validate
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

parser = argparse.ArgumentParser()
parser.add_argument('--features_csv', required=True)
args = parser.parse_args()

df = pd.read_csv(args.features_csv, index_col=0)
# Expect a column named 'diagnosis' with values like 'AD' or 'CN'
if 'diagnosis' not in df.columns:
    raise SystemExit("features CSV must contain a 'diagnosis' column")

X = df.drop(columns=['diagnosis'])
y = df['diagnosis'].map({'CN':0, 'AD':1}).astype(int)

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', RandomForestClassifier(n_estimators=200, random_state=42))
])
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scoring = ['accuracy', 'roc_auc']
res = cross_validate(pipe, X, y, cv=cv, scoring=scoring)
print('CV results:')
for k, v in res.items():
    if k.startswith('test_'):
        print(f"{k}: mean={v.mean():.3f}, std={v.std():.3f}")
```

4) README snippet to reproduce

1. Prepare BIDS dataset under /path/to/bids
 2. Edit run_fmriprep.sh with correct paths and run for a small subset:
bash scripts/run_fmriprep.sh sub-001 sub-002
 3. After fMRIPrep finishes, run FreeSurfer recon or use freesurfer outputs from derivatives.
 4. Extract volumes:
python scripts/extract_volumes.py --freesurfer_dir derivatives/freesurfer --out results/aseg_volumes.csv
 5. Combine with participants.tsv to add diagnosis column and run classification:
python scripts/classification_pipeline.py --features_csv results/features_with_labels.csv
-